

1) Диаграмма Нэсси–Шнейдермана (структурограмма)

Идея. Алгоритм изображается **вложенными блоками** без стрелок. Каждая конструкция структурного программирования — это прямоугольный блок или композиция прямоугольников.

Базовые элементы

- **Последовательность** — вертикально расположенные блоки один за другим.
- **Ветвление if/else** — прямоугольник, разделённый на секции «да/нет».
- **Множественный выбор (switch/case)** — блок, разбитый на несколько секций по вариантам.
- **Цикл** — блок с меткой (пока/до), охватывающий вложенный набор действий.
- **Подпрограмма/вызов** — именованный вложенный прямоугольник.

Сильные стороны

- Читаемость для обучения: нет путаницы со стрелками, структура очевидна.
- Подталкивает к **одному входу и одному выходу** у каждой конструкции.

Ограничения

- Плохо показывает **параллелизм** и **потоки данных**.
- Большие алгоритмы становятся громоздкими.

Где применять: обучение, проектирование процедурной логики, пояснение алгоритмов без акцента на данные или компоненты системы.

2) ДРАКОН-схемы

Идея. Визуальный язык, созданный для **когнитивной эргономики**: схема читается сверху вниз как «река времени», элементы строго выровнены, минимизировано пересечение линий.

Алфавит и правила (в упрощённом виде)

- **Заголовок и конец** — рамочные иконки, формируют «скелет».
- **Действие** — прямоугольник с текстом шага.
- **Развилка/слияние** — строго оформленные узлы для ветвлений.
- **Повторы** — типовые шаблоны для «пока», «до», счётных циклов.
- **Подсхемы** — вызовы подпрограмм («иконки») и декомпозиция.

- Жёсткие **правила верстки**: вертикальный основной поток, аккуратные боковые ответвления, отсутствие «паутины» стрелок.

Сильные стороны

- Очень **читабельно** для сложных алгоритмов и инструкций.
- Подходит для регламентов, медицинских алгоритмов, учебных материалов.

Ограничения

- Требует дисциплины/редактора ДРАКОН.
- Не заменяет нотации уровня архитектуры или данных.

Где применять: пошаговые алгоритмы, регламенты, инструкции, сценарии (включая чат-ботов/голосовых ассистентов).

3) UML Activity (диаграмма деятельности)

Идея. Поведенческая диаграмма UML, описывает **поток работ/действий** и правила их переходов.

Базовые элементы

- **Начало** (закрашенный кружок) и **конец** (бычий глаз или пустой кружок с обводкой).
- **Действие/деятельность** — прямоугольник со скруглёнными углами.
- **Потоки управления** — стрелки между действиями.
- **Решение/слияние** — ромбы с пометками условий.
- **Параллелизм** — **fork/join** (толстые чёрточки) для разветвления/синхронизации.
- **Объектные потоки** — стрелки с прямоугольниками объектов (данные на вход/выход действия).
- **Дорожки ответственности (swimlanes)** — разделение по ролям/подсистемам.

Сильные стороны

- Хорошо показывает **ветвления, параллельность, ответственных**.
- Встраивается в экосистему UML (use case, state, sequence и др.).

Ограничения

- На детальном уровне может перегружаться.
- Не описывает структуру данных лучше специализированных нотаций.

Где применять: бизнес-процессы, сценарии использования, алгоритмы операций классов, согласование потоков между ролями.

4) DFD — диаграмма потоков данных

Идея. Показывает, как данные текут через систему: от внешних сущностей к процессам, в хранилища и обратно.

Базовые элементы (Yourdon–DeMarco / Gane–Sarson)

- **Процесс** — преобразует входные потоки в выходные.
- **Поток данных** — именованная стрелка.
- **Хранилище данных** — пассивный накопитель (файл/БД).
- **Внешняя сущность** — источник/приёмник данных вне системы.

Уровни

- **Контекстная диаграмма (Level 0)** — система как один процесс + основные внешние сущности и потоки.
- **Декомпозиция (Level 1, 2, ...)** — «взрыв» процессов на более детальные подпроцессы с **балансировкой** потоков между уровнями.

Сильные стороны

- Фокус на данных и их маршрутах; наглядна для аналитики.

Ограничения

- Слабее показывает логику управления и состояния.
- Требуется аккуратной декомпозиции и единообразной нотации.

Где применять: анализ требований к ИС, интеграционные потоки, границы системы.

5) Р-схемы (R-charts)

Идея. Структурная запись алгоритма в виде **ориентированного графа**, построенного из типовых подграфов с **единственным входом и выходом**. Рёбра (дуги) могут быть подписаны условиями/метками.

Типовые конструкции

- **Последовательность** — линейное соединение подграфов.
- **Ветвление** — разветвлённые дуги с подписями условий.
- **Цикл** — обратная дуга к началу подграфа с условием продолжения/выхода.
- **Подпрограмма** — именованный подграф с одним входом/выходом.

Сильные стороны

- Формальная строгость, удобство доказательств корректности.
- Естественно выражает **структурное программирование**.

Ограничения

- Реже встречается в прикладных инструментах.
- При большом числе ветвей «граф» может утяжеляться.

Где применять: учебные курсы по алгоритмам, формальная проверка логики, сопоставление с текстовой программой.

6) Языки для программирования ПЛК (IEC 61131-3): SFC и FBD

Контекст. Оба языка входят в стандарт **IEC 61131-3** для ПЛК. SFC описывает последовательности состояний/шагов, FBD — потоки данных через функциональные блоки.

SFC — Sequential Function Chart

Идея. Модель процесса как **шаги (steps)** и **переходы (transitions)**; к шагам привязаны действия (**actions**). Подходит для последовательных операций и может поддерживать параллельные ветви.

Базовые элементы

- **Начальный шаг** — единственный старт.
- **Шаг** — прямоугольник с именем состояния; к шагу привязываются действия (выполняются, пока шаг активен).
- **Переход** — горизонтальная черта с **логическим условием** включения следующего шага.
- **Разветвление/синхронизация** — параллельные шаги и переходы (AND/OR-ветвления по условию).
- **Поддиаграммы/макрошаги** — декомпозиция для сложных последовательностей.

Сильные стороны

- Естественно выражает **последовательности и межэтапные условия**.
- Снижает количество «ручных» переменных состояния.
- Наглядна для пуско-наладочных и циклических режимов машин.

Ограничения

- Большие деревья решений выражать неудобно — лучше комбинировать с ST/FBD.
- Вычислительные алгоритмы и интенсивная логика — не основная сильная сторона.

Где применять: этапы технологических процессов, автоматы состояний, последовательности запусков/остановов, рецептурные шаги.

FBD — Function Block Diagram

Идея. Функциональные блоки с входами/выходами соединяются **линиями**; получается **граф потока данных**. Блоки могут иметь внутреннюю память (экземпляры) и повторно использоваться.

Базовые элементы

- **Функциональный блок/функция** — «чёрный ящик» (например, таймер TON, триггер, ПИД).
- **Соединения** — линии между выходами и входами; допускается ветвление/объединение сигналов.
- **Сетевые участки (networks)** — логические фрагменты схемы.
- **Переменные/константы** — источники/приёмники значений.

Сильные стороны

- Отлично показывает **обработку сигналов и взаимосвязи данных**.
- **Повторное использование** через экземпляры блоков; просто комбинировать готовые функции.
- Хорош для непрерывного управления, интерлоков, ПИД-контуров.

Ограничения

- Труднее читать сложные **последовательности** и «временные» сценарии — лучше вместе с SFC.
- При множестве ветвлений схема быстро разрастается.

Где применять: логика межблоковых связей, обработка аналоговых/дискретных сигналов, библиотечные контроллеры (таймеры, счётчики, ПИД и т.п.).

Мини-шпаргалка (ПЛК)

- **SFC:** старт → шаги → переходы (условия); возможен параллелизм; действия привязаны к шагам.
- **FBD:** блоки + связи; данные текут **от выходов к входам**; удобно для повторного использования функций.

7) Нотации бизнес-процессов: IDEF0 и BPMN

IDEF0 — функциональное моделирование (SADT)

Идея. Фокус на **функциях** и их интерфейсах. Каждый блок описывает, *что* делается, а стрелки показывают **ICOM**: **I**ntput (слева), **C**ontrol (сверху), **O**utput (справа), **M**echanism (снизу).

Базовые элементы

- **Функция (бокс)** — действие в форме глагольной фразы, имеет номер (А-номер).

- **Стрелки ICOM** — входы, управляющие воздействия/ограничения, выходы, механизмы/ресурсы.
- **Контекстная диаграмма A-0** — определяет границы модели.
- **Иерархическая декомпозиция** — $A_0 \rightarrow A_1 \dots A_n$, с **балансировкой** интерфейсов между уровнями.

Сильные стороны

- Чёткая **граница и интерфейсы** процесса; отлично для верхнего уровня.
- Методически поддерживает **top-down** анализ.

Ограничения

- Не показывает детальную **управляющую логику/временную последовательность**.
- Для событийного взаимодействия и исключений лучше BPMN.

Где применять: архитектурные модели предприятия/систем, определение области и интерфейсов, предпроектный анализ.

BPMN 2.0 — нотация бизнес-процессов

Идея. Стандарт OMG/ISO для описания **потока работ**, взаимодействия участников и обработок событий. Богатая нотация и связь с автоматизацией.

Базовые элементы

- **Объекты потока (Flow Objects): События** (старт/промежуточные/конец), **Задачи/Подпроцессы**, **Шлюзы** (XOR/OR/AND, event-based и др.).
- **Связующие: Последовательный поток** (sequence flow), **Сообщения** (message flow), **Ассоциации**.
- **Дорожки ответственности: Пулы/Лейны** (организации/роли).
- **Артефакты: Данные/Хранилища**, группы, аннотации.

Сильные стороны

- Выражает **варианты, исключения и взаимодействие** через сообщения.
- Подходит для **автоматизации** (BPMN 2.0 вводит исполнение и формальные семантики).

Ограничения

- Большой алфавит символов: нужна моделерская дисциплина и стандарты.
- Не заменяет **модели данных/организации**.

Где применять: операционные процессы, регламенты, автоматизация и интеграции между подразделениями/системами.